Step-by-Step Implementation Guide

FastAPI + SQLite + JavaScript (Your Exact `app.js` Code)

This guide explains:

1. What each part of your JavaScript does
2. How to connect it to FastAPI + SQLite
3. How to run the full system
4. How to verify data is stored in the database

PART 1 – Understanding Your JavaScript Code

STEP 1 – Selecting HTML Elements

```
const emailInput = document.getElementById("email");
```

What This Does

- Finds the HTML input field with id="email"
- Stores it in a variable
- Allows JavaScript to read the value inside it

This connects JavaScript to the webpage.

Same logic applies to:

- firstNameInput
- lastNameInput
- cityInput
- countryInput
- Buttons
- productContainer

Without this step, JavaScript cannot interact with the page.

SECTION A – Register Customer (POST Request)

```
registerBtn.addEventListener("click", function () {
```

What This Does

- Listens for when the Register button is clicked
- Runs the function when clicked

Creating Customer Object

```
const customerData = {
    email: emailInput.value,
    first_name: firstNameInput.value,
    last_name: lastNameInput.value,
    city: cityInput.value,
    country: countryInput.value
};
```

What This Does

- Reads user input values
- Creates a JavaScript object
- Keys must match backend expectations

This object is still JavaScript format — not JSON yet.

Sending Data to FastAPI

```
fetch("http://127.0.0.1:8000/customers", {
    method: "POST",
    headers: {
        "Content-Type": "application/json"
    },
    body: JSON.stringify(customerData)
})
```

What This Does

- Sends request to backend
- Method POST = sending data
- JSON.stringify converts JavaScript object into JSON string
- Headers tell FastAPI it is receiving JSON

Handling Response

```
.then(response => response.json())
```

Converts server response into usable JavaScript.

After Success

```
alert("Customer registered successfully!");
```

Shows confirmation.

Then clears the form:

```
emailInput.value = "";
```

SECTION B – Load All Products

```
loadProductsBtn.addEventListener("click", function () {
```

Runs when Load button is clicked.

Fetch Products

```
fetch("http://127.0.0.1:8000/products")
```

This sends GET request.

GET means: retrieve data.

Convert Response

```
.then(response => response.json())
```

Server sends JSON → Convert to JavaScript array.

Display Products

```
displayProducts(data);
```

Calls reusable function.

SECTION C – Filter by Category

Reads category input:

```
const categoryId = categoryIdInput.value;
```

Builds dynamic URL:

```
/products?category_id=2
```

FastAPI receives query parameter and filters SQL.

DISPLAY FUNCTION

```
function displayProducts(products)
```

What It Does

1. Clears old results:
2. Loops through array:
3. Creates new div
4. Inserts product values into HTML
5. Appends to page

This is dynamic page rendering.

PART 2 – Backend Setup (FastAPI + SQLite)

Create:

```
api_app.py
```

Make sure it contains:

- POST /customers
- GET /products
- SQLite connection

- CORS middleware enabled

PART 3 – How To Execute the Full System

STEP 1 – Start Backend

In project folder:

```
uvicorn api_app:app --reload
```

You should see:

```
Uvicorn running on http://127.0.0.1:8000
```

Test:

Open browser:

```
http://127.0.0.1:8000/products
```

You should see JSON.

If yes → Backend working.

STEP 2 – Run Frontend

Go into your static folder:

```
cd static
```

Run:

```
python -m http.server 5500
```

Open browser:

```
http://127.0.0.1:5500
```

Now frontend is running.

PART 4 – How Data Appears in Database

When you fill form and click Register:

1. JavaScript collects input values
2. Converts to JSON
3. Sends POST request
4. FastAPI receives JSON
5. SQLite executes:

```
INSERT INTO customers ...
```

6. Database writes data
7. Response sent back
8. Alert appears

PART 5 – How To Check Data Is Stored

Option 1 – Use Python

Open terminal:

```
python
```

Then:

```
import sqlite3

conn = sqlite3.connect("local_retail.db")
cursor = conn.cursor()

cursor.execute("SELECT * FROM customers")
print(cursor.fetchall())

conn.close()
```

You should see your entered data.

Option 2 – Use SQLite Viewer

Open database file in:

- DB Browser for SQLite (recommended)

- VS Code SQLite extension

Open `local_retail.db`
Open customers table
Check rows

PART 6 – How To Know Everything Works

✓ Form submits without refresh
✓ Alert appears
✓ Data visible in SQLite
✓ Load Products button shows products
✓ Filter works
✓ No console errors

Press F12 → Console to check.

Complete Flow Summary

User enters form
↓
JavaScript creates object
↓
Fetch POST
↓
FastAPI receives JSON
↓
SQLite inserts row
↓
FastAPI sends response
↓
JavaScript updates page

You have built a real working API-driven retail system.